Visualizador de un UR3e por TCP/IP

Angel Yesid Ardila Ardila · Universidad de Deusto · Octubre de 2026

Todo el código del repositorio disponible en: https://github.com/angeloob11/TCP_ER_SERVER

1. Estructura del código

El proyecto corresponde a cliente TCP escrito en Python que lee el estado de uno o varios robots UR3e y crea un gemelo digital que dibuja los robots. los script usados se resumen en (tabla 1).

Tabla 1: Scripts del proyecto y su responsabilidad.

Módulo	Responsabilidad
`buffer_ur.py`	Función de conexión y manejo de errores en caso de fallos de red. Función de lectura de datos y función de escritura y decodificación de datos.
`primaria.py`	Lector del puerto 30001: lee los mensajes de tipo 16 extrae la posición de los eslabones y el estado del robot y tipo 20 (errores).
`tiempo_real.py`	Lector del puerto 30003 por desplazamientos fijos, con medida de paquetes/s.
`multi_robot.py`	Maneja varios robots desde `config/robots.json`, un hilo y un `Lock` por robot, y maneja la visualización.
`rx.py`	Diagnóstico sin hilos: compara el reloj del robot con el del PC para medir la tasa real.
`constantes.py`	Constantes necesarias para el proyecto.

La arquitectura (figura 1) separa la recepción, con un hilo lector por robot que consume todo lo que llega y guarda solo el último estado para la simulación, un bucle principal a 30 Hz que copia ese estado y mueve el robot de del gemelo digital. Así la interfaz de 500 Hz no acumula retraso por culpa del renderizado. Las órdenes URScript se envían por el mismo socket que usa el lector: como ese hilo nunca deja de leer, el buffer de recepción del sistema operativo no se llena y la conexión no se bloquea.

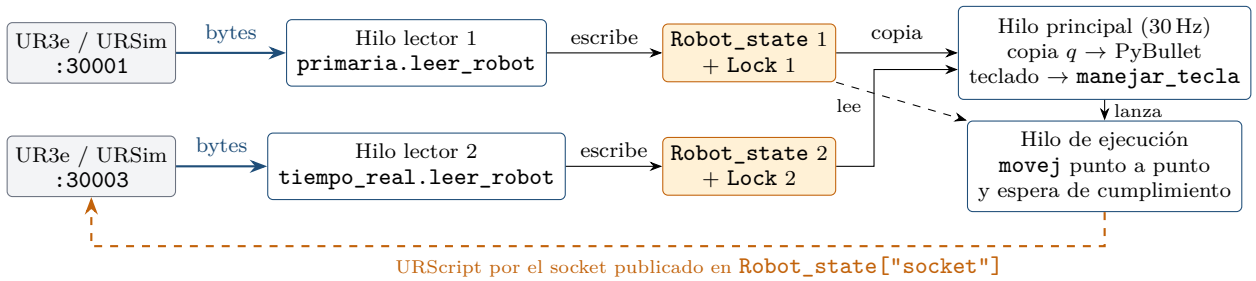


Figura 1: Hilos y datos compartidos. Cada robot tiene su propio hilo, estado y `Lock`, así que la caída de uno no afecta a los demás.

Para garantizar la conectividad, `conectar_con_reintento()` reintenta cada segundo ante cualquier `OSError` (puerto cerrado, IP que no responde, red inalcanzable) y aplica un *timeout* de recepción: 2 s en la primaria y 1 s en tiempo real. Si el servidor desaparece sin enviar FIN ni RST, el silencio provoca un `TimeoutError` y no un `recv()` bloqueado para siempre. Cualquier error del lector (`OSError`, `ValueError` por desalineación o `struct.error`) cierra el socket, pone `Robot_state["socket"]=None` y vuelve a conectar.

2. Estrategia de lectura: el buffer de datos

TCP entrega un flujo de bytes, no mensajes: `recv(4096)` devuelve lo que haya llegado hasta ese momento, que puede ser medio mensaje, uno entero o varios seguidos (sección 6). Por eso ningún lector trata lo devuelto por `recv()` como un mensaje. Los bytes se acumulan en un `bytearray` y se cortan usando la longitud con la que empieza cada mensaje. El mismo código sirve para las dos interfaces; solo cambia el tamaño mínimo de cabecera (5 bytes en 30001, con longitud y tipo; 4 bytes en 30003, solo longitud).

src/buffer_ur.py, líneas 25–37

```python
def extraer_mensajes(buffer: bytearray, tam_cabecera: int) -> list[bytes]:
    mensajes = []
    while len(buffer) >= tam_cabecera:                    # (1) hay cabecera completa?
        (longitud,) = struct.unpack_from(">i", buffer, 0)  # (2) leer L sin consumirla
        if longitud < tam_cabecera or longitud > LONGITUD_MAXIMA:   # 64 kB
            raise ValueError(f"Longitud imposible: {longitud} (flujo desalineado)")
        if len(buffer) < longitud:                        # (3) mensaje incompleto
            break                                         #     -> esperar al siguiente recv()
        mensajes.append(bytes(buffer[:longitud]))         # (4) cortar exactamente L bytes
        del buffer[:longitud]                             #     y conservar el sobrante
    return mensajes                                       # (5) puede devolver 0, 1 o N mensajes
```

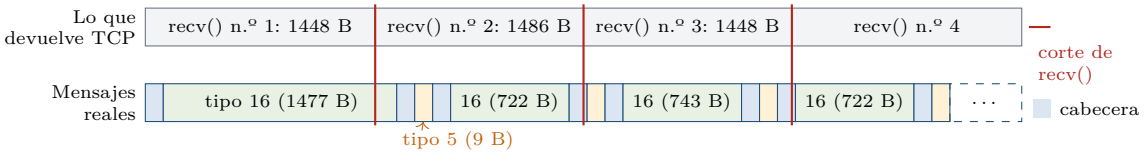


Figura 2: Los cortes de `recv()` (en rojo) no coinciden con los límites de los mensajes. El buffer guarda el trozo que sobra de cada `recv()` y lo completa con el siguiente.

Características del buffer:

- La longitud se lee *sin consumirla* (`unpack_from`) y solo se borra del buffer cuando el mensaje está completo; el sobrante se queda para la siguiente vuelta.
- Una longitud menor que la cabecera o mayor que 64 kB solo puede significar que el flujo está desalineado. En ese caso no se intenta resincronizar: se lanza `ValueError`, el lector cierra la conexión y abre una nueva, que siempre empieza en un límite de mensaje.
- Si `recv()` devuelve 0 bytes, el robot ha cerrado la conexión, y se trata como tal.
- En tiempo real la longitud tampoco es una constante: se lee de cada paquete, de modo que el mismo código funciona con versiones que envían 1140 o 1220 bytes.

3. Decodificación de los mensajes

Interfaz primaria (30001). Cada mensaje cortado se identifica por su tipo (byte 4). Del tipo 16 (*Robot State*) se recorren los subpaquetes con `offset += longitud_sub`, sin suponer ningún orden. El tipo 20 (*Robot Message*) solo se decodifica cuando `robotMessageType = 6` (código de error). Cualquier otro tipo se ignora: su longitud ya ha permitido saltarlo.

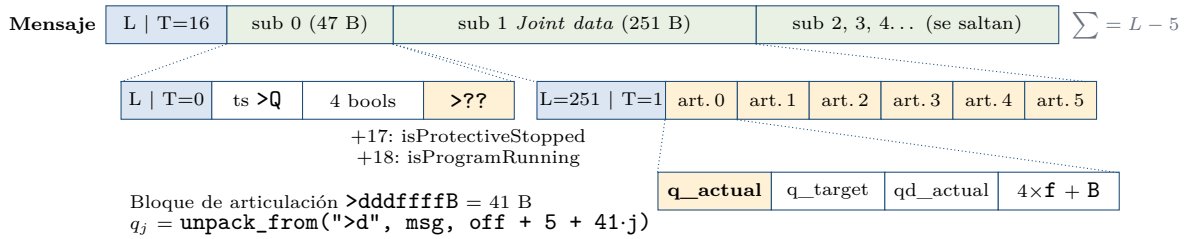


Figura 3: Anidamiento mensaje → subpaquete → articulación en la interfaz primaria, con los campos que lee el programa.

src/primaria.py, recorrido de subpaquetes del tipo 16 (simplificado)

```python
offset, fin = 5, min(longitud, len(mensaje))
while offset + 5 <= fin:
    longitud_sub, tipo_sub = struct.unpack_from(">iB", mensaje, offset)
    if longitud_sub < 5 or offset + longitud_sub > fin:      # se sale del mensaje: corrupto
        raise ValueError(f"Subpaquete tipo {tipo_sub} con longitud inválida {longitud_sub}")
    if tipo_sub == 0:   ...  # isProtectiveStopped, isProgramRunning en offset+17
    elif tipo_sub == 1: q = [struct.unpack_from(">d", mensaje, offset + 5 + 41*i)[0] for i in range(6)]
    offset += longitud_sub                                   # saltar al siguiente subpaquete
```

Interfaz de tiempo real (30003). Hay un único tipo de paquete con los campos en posiciones fijas, así que basta con cortar el paquete y saltar al desplazamiento de cada campo (tabla 2). Los campos de estado solo se leen si el paquete es lo bastante largo; con versiones antiguas quedan a `None` en lugar de leer bytes ajenos.

Tabla 2: Campos de tiempo real usados (hoja *Client Interfaces*, SW 5.x; desplazamiento $= 4 + 8 \cdot (\text{índice} - 1)$).

Byte	Campo	Formato	Uso en el programa
0	Message size	`>i`	delimitar el paquete en el buffer (1220 B en URSim 5.25)
4	Time	`>d`	reloj del controlador (`rx.py` mide con él la tasa real)
252	q actual	`>6d`	ángulos de las 6 articulaciones
812	Safety mode	`>d`	3 = parada de protección; 5–9 se tratan como error
1052	Program state	`>d`	2 = programa en ejecución (equivale a `isProgramRunning`)

4. El estado compartido y el `Lock`

Cada robot tiene un diccionario `Robot_state` (`q`, `isProgramRunning`, `isProtectiveStopped`, `error`, `socket`) y su propio `threading.Lock`. El criterio es que el lock solo protege copias. El lector decodifica "fuera" del lock y solo lo toma para asignar los valores ya calculados. Quien consume (el bucle de dibujo, `guardar_posicion` o `esperar_movimiento`) lo toma para copiar (`q[:]`) y lo suelta antes de usar los datos. Así las secciones críticas duran microsegundos y el lector de 500 Hz nunca espera a PyBullet.

```
# Hilo lector (tiempo_real.py): un único bloque por paquete -> los cuatro campos son coherentes
with lock:
    Robot_state["q"] = list(q);  Robot_state["isProgramRunning"] = program_running
    Robot_state["isProtectiveStopped"] = protective_stopped;  Robot_state["error"] = error
# Hilo principal (multi_robot.py): copiar bajo el lock y dibujar fuera de él
with r["lock"]:
    q = r["Robot_state"]["q"][:]
sim.move_robot(r["id"], q)
```

Un lock *por robot*, y no uno global, evita que un robot lento o desconectado retenga a los demás. `q` se sustituye por una lista nueva en vez de modificarse elemento a elemento, de modo que ni siquiera un lector sin lock vería una lista a medias. El lock sigue siendo necesario para que `q` y los indicadores de estado que se leen juntos procedan del mismo mensaje.

5. Envío de órdenes (enseñar y reproducir)

`enviar_movimiento()` valida que haya 6 ángulos finitos dentro de $\pm 2\pi$ (para no enviar grados por error), limita v y a a 0,3 rad/s y 0,5 rad/s^2 y envía el programa del robot con`def mover(): movej([...], a, v) end`.

`esperar_movimiento()` deduce el cumplimiento del estado publicad,: espera a que `isProgramRunning` pase a verdadero (si no ocurre en 2 s, el robot ha rechazado la orden por parte del programa), vigila errores y paradas de protección mientras dura el movimiento, y al terminar exige que la desviación sea menor de 0,01 rad en cada articulación. La serie se ejecuta punto a punto en un hilo aparte, lo que permite cancelarla (`stopj`) y reanudarla desde el punto que falló. Los puntos se guardan en `config/puntos.json`.

6. Respuestas a las preguntas del apartado 12

P1. *Registrad durante un minuto cuántos bytes devuelve cada llamada a recv(). ¿Coincide siempre con la longitud de un mensaje? ¿Qué consecuencia tiene eso para vuestro código?*

NO, En la primaria, el robot escribe el mensaje de estado (722 B) y justo después el de tipo 5 (9 B), así que el caso habitual es un `recv()` de 731 B con *dos* mensajes dentro. El primer mensaje de estado (1477 B) es mayor que un segmento Ethernet (MSS $\approx$ 1448 B) y llega partido en 1448 + 38 B, es decir, como un mensaje incompleto. En tiempo real, el 99,8 % de los `recv()` devuelve justo 1220 B, pero aparecen 2440, 3660 y 4096 B. Este último caso es el peligroso: $4096 = 3 \times 1220 + 436$, o sea tres mensajes y un trozo del cuarto. Si el consumidor se retrasa, el 97 % de las llamadas devuelve 4096 B y en el 99,6 % queda un trozo en el buffer.

P2. *¿Por qué el mensaje y cada subpaquete empiezan con su longitud? ¿Qué podría hacer vuestro programa con un subpaquete de un tipo que no aparece en la documentación?*

Por dos motivos. Primero, *delimitar*: TCP no conserva los límites de los mensajes, así que la longitud es la única forma de saber dónde acaba uno y empieza el siguiente. Segundo, *extensibilidad*: con la longitud se puede saltar un bloque sin entender lo que contiene (`siguiente = inicio + longitud`). UR puede añadir subpaquetes o alargar los existentes en versiones nuevas sin romper a los clientes antiguos. Un subpaquete de tipo desconocido simplemente se salta; así lo hace nuestro bucle, que solo actúa sobre los tipos 0 y 1. Lo único que se comprueba es que su longitud no se salga del mensaje: si se sale, el mensaje se descarta como corrupto.

P3. *Si decodificáis `q_actual` sin indicar big-endian, ¿qué valores obtenéis? Explicad por qué con un ejemplo de 8*

bytes

El robot envía los números en orden de red (big-endian, el byte más significativo primero) y un PC x86 es little-endian. Sin `>`, `struct` lee los 8 bytes en orden inverso y los interpreta como otro número completamente distinto. Signo, exponente y coeficiente salen de bytes que no les corresponden:

q enviado (rad)	8 bytes en el cable (`>d`)	Leído con `<d`	Leído con `>d`
−1,6007	`bf f9 9c 77 9a 6b 50 b1`	$-3{,}72 \times 10^{-71}$	−1,6007
1,5708 ($\pi/2$)	`3f f9 21 ff 2e 48 e8 a7`	$-1{,}93 \times 10^{-116}$	1,5708
3,14159	`40 09 21 f9 f0 1b 86 6e`	$2{,}56 \times 10^{224}$	3,14159
0,0	`00 00 00 00 00 00 00 00`	0,0	0,0

Salen valores casi nulos o astronómicos (del orden de $10^{\pm 300}$). Hay un caso traicionero: con el robot en la postura cero todos los bytes son `00` y el error no se nota hasta que se mueve. La longitud también se corrompe: 1220 (`00 00 04 c4`) leído con `<i` da −1 006 370 816. Nuestra validación de longitud cortaría la conexión de inmediato.

P4. *Comparad la interfaz primaria y la de tiempo real: ¿qué ventaja tiene cada una? ¿Cuál elegiríais para un visualizador que alimenta un panel de supervisión? ¿Y para registrar la trayectoria de un movimiento rápido?*

Para un pane de supervisión la red primaria (10 Hz, mensajes variables con subpaquetes) ya que es extensible y además de las posiciones trae información de supervisión: modo del robot, paradas de protección, errores (tipo 20), versión y excepciones de programa. Para el caso de registrar un movimiento rápido, la red de tiempo real (500 Hz, paquete fijo) es sencilla y rápida de decodificar por desplazamientos, y su resolución temporal es 50 veces mayor, pero no trae mensajes de evento y su formato depende de la versión, en comparación a 10 Hz, un eje a 3 rad/s recorre 0,3 rad entre dos muestras, mientras que a 500 Hz solo recorre 6 mrad.

P5. *¿Por qué en NAT no podéis conectaros a URSim desde el PC sin configurar nada más, y en modo puente sí? ¿Qué equipo de la red ve la dirección MAC de vuestra VM?*

En NAT, VirtualBox coloca la VM en una red privada interna (10.0.2.x) detrás de un router virtual. Las conexiones saliente de la VM se traducen a la IP del PC, pero nadie puede iniciar una conexión entrante hacia el puerto 30001 de la VM sin una regla de reenvío de puertos: la IP de la VM no es alcanzable desde fuera. En modo puente, la VM se conecta directamente a la red física con su propia MAC y obtiene una IP de la misma subred (10.172.18.x en el aula), igual que cualquier otro equipo. La MAC de la VM la ven los equipos del mismo dominio de difusión: el switch, que la aprende en su tabla, el PC anfitrión y los demás equipos de la LAN, en sus tablas ARP. No la ven los equipos que están detrás de un router, porque el router reescribe las cabeceras Ethernet en cada salto.

P6. *Desconectad el cable de red del PC durante 5 segundos y volved a conectarlo. ¿Qué hace vuestro programa? ¿Qué haría TCP con los datos durante ese tiempo?*

TCP: un corte físico no cierra la conexión, porque no se envía ni FIN ni RST y el robot sigue escribiendo en su buffer de envío. Al reconectar el cable, si la conexión sigue viva, entrega los datos pendientes en orden y sin pérdidas. Llegan de golpe y atrasados, y si el buffer del robot se ha llenado, este puede dejar de escribir o cerrar la conexión.

En el programa cuando se deja de detectar la conexión, el *timeout* de recepción salta (1 s en tiempo real, 2 s en la primaria) y lanza `TimeoutError`. El lector lo captura, marca el socket como `None`, cierra y reintenta cada segundo. El robot dibujado se queda en la última postura conocida y, en el multi-robot, los demás robots siguen moviéndose. Al volver la red, se abre una conexión nueva que empieza alineada y con datos frescos, sin la cola atrasada. Si había una serie en ejecución, se detiene por falta de conexión y se reanuda con `c`. La figura 4 muestra este comportamiento con URSim.

```
Conectado a 192.168.1.140:30001
Recibido mensaje tipo 20 con longitud 55
Recibido mensaje tipo 20 con longitud 24
Recibido mensaje tipo 24 con longitud 181
Recibido mensaje tipo 23 con longitud 881
Recibido mensaje tipo 16 con longitud 1477
Estados de las articulaciones: [ -91.71
  -98.96 -126.22 -46.29  91.39  -1.78]
Recibido mensaje tipo 5 con longitud 9
Recibido mensaje tipo 16 con longitud 743
```

```
Tasa de recepcion: 377.00 paquetes/s
Tasa de recepcion: 371.00 paquetes/s
Tasa de recepcion: 308.00 paquetes/s
Conexion perdida: TimeoutError('timed out').
  Reconectando...
Servidor 192.168.1.140:30003 no disponible
  (intento 1): timed out. Reintentando en 1.0 s...
Servidor 192.168.1.140:30003 no disponible
  (intento 2): timed out. Reintentando en 1.0 s...
```

Figura 4: Capturas de consola con URSim. Izquierda: `primaria.py` en modo de depuración, con mensajes de tipos y longitudes variables. Derecha: `tiempo_real.py` al apagar URSim: *timeout*, reconexión cada segundo y sin bloqueo.

P7. *Medid el retraso entre el movimiento de URSim y el del robot simulado con la interfaz primaria y con la de tiempo real. ¿De qué depende?*

El retraso es la suma de:

1. el *periodo de muestreo*, de 0 a 100 ms en la primaria y de 0 a 2 ms en tiempo real

2. la red y la pila TCP, por debajo de 1 ms en una LAN
3. el bucle de dibujo a 30 Hz, de 0 a 33 ms
4. el refresco de pantalla. Estimado: unos 50–130 ms con la primaria frente a unos 20–40 ms con tiempo real.

La primaria va visiblemente detrás. El factor crítico es que el consumidor siga el ritmo del productor. Con un consumidor lento los datos procesados llegaban cada vez más viejos porque la cola de TCP no deja de crecer. Con lectura continua, el retraso se mantuvo en torno a 1 ms. Por eso la recepción va en un hilo que no dibuja. Con URSim medimos entre 245 y 377 paquetes/s mientras PyBullet dibujaba, y unos 450/s con `rx.py`, que compara el reloj del robot con el del PC para separar lo que el simulador envía de lo que nuestra aplicación consume.

P8. *¿Qué ocurriría si el bucle de PyBullet leyera el estado compartido sin Lock mientras un hilo lo está escribiendo?*

Leería un estado inconsiste: mezcla de dos mensajes distintos. Lo comprobé con un hilo que imita al lector (300 000 escrituras) y otro que imita a `esperar_movimiento`, forzando cambios de hilo frecuentes (`sys.setswitchinterval(1e-6)`) para simular carga:

Forma de compartir el estado	q con ángulos de dos mensajes	q y `isProgramRunning` de mensajes distintos
Sin lock, q modificado elemento a elemento	57,5 % de las lecturas	5,1 %
Lock por campo (secciones separadas)	0 %	64,5 %
Un lock por mensaje (como en `tiempo_real.py`)	**0 %**	**0 %**

En el visualizador, el efecto sería un parpadeo o una postura que nunca existió, en cuanto al control, si `esperar_movimiento` leyera `isProgramRunning = False` del mensaje nuevo junto con un q del anterior, daría por terminado o por fallido un movimiento que no lo está. El GIL de CPython hace atómica una asignación individual, pero no un grupo de lecturas o escrituras. El lock garantiza esa atomicidad de grupo. Los porcentajes están inflados por la prueba de estrés: a 500 Hz la ventana es de microsegundos y el fallo es raro, pero precisamente por eso es difícil de depurar.

P9. *¿Por qué no basta con haber enviado un movej para saber que el robot ha llegado al punto? ¿Qué información del propio robot usáis para confirmarlo?*

`sendall()` solo garantiza que los bytes han salido de nuestro buffer, y el ACK de TCP, que han llegado a la pila del controlador, ninguno dice si el programa se ha aceptado, ni si se ha ejecutado o completado. El robot puede rechazarlo (por ejemplo si no está en modo remoto), abortarlo por una parada de protección o detenerlo por otro programa que llegue después. con `isProgramRunning` (subpaquete 0, o `program_state` = 2 en tiempo real) se confirma si el robot recibe el programa cuando pasa a verdadero en menos de 2 s y vuelve a falso, y durante ese tiempo no hay `isProtectiveStopped` (o `safety_mode` = 3) ni mensajes de error tipo 20 / subtipo 6.

P10. *Cualquier equipo de la red puede conectarse al puerto 30001 y enviar un programa al robot. ¿Qué riesgo supone en una planta real? Proponed dos medidas, una en la configuración del robot y otra en la arquitectura de red.*

Cualquier equipo de la red de planta, o un atacante que haya entrado en ella, podría mover un robot con personas cerca, cambiar programas o velocidades, provocar paradas (denegación de servicio) o leer información del proceso. No hay autenticación ni cifrado: el URScript viaja en texto plano.